

SupplyFlow Application Security & VAPT Assessment

Security Assessment Report

Application: SupplyFlow ERP / Business Management System

Assessment Type: Application Security Testing & VAPT

Prepared by: Subrat Gupta

Technology Stack: React, TypeScript, Node.js, Express, PostgreSQL, Prisma

Document Control

Document Title	SupplyFlow Application Security & VAPT Assessment
Application	SupplyFlow ERP / Business Management System
Assessment Type	Application Security / VAPT
Prepared By	Subrat Gupta
Assessment Environment	Local Development Environment
Report Version	1.0
Date	October 2026

Contents

1	Executive Summary	6
2	Application Overview	8
2.1	SupplyFlow Overview	8
2.2	Technology Stack	8
2.3	Application Architecture	9
2.4	Security-Relevant Components	10
3	Assessment Scope	12
3.1	Scope	12
3.2	In-Scope Components	12
3.3	Assessment Environment	13
3.4	Out of Scope	13
3.5	Testing Limitations	14
4	Security Testing Methodology	15
4.1	Testing Approach	15
4.2	Manual API Security Testing	15
4.3	Source Code Review	16
4.4	Dependency Vulnerability Assessment	17
4.5	Security Header Assessment	17
4.6	API Response Data Assessment	18
4.7	Remediation and Retesting	18
4.8	Tools Used	19
5	Authentication and Authorization Assessment	20
5.1	JWT Authentication	20
5.2	Authentication Test Results	20
5.2.1	Unauthenticated Request	21
5.2.2	Invalid JWT	21
5.2.3	Valid Authentication	21

5.3	Role-Based Access Control	22
5.4	Authorization Test Results	22
5.4.1	Admin-Only Inventory Modification	23
5.4.2	Admin-Only Sales Order Operations	23
5.5	Authentication and Authorization Conclusion	24
6	API Security Assessment	25
6.1	API Security Overview	25
6.2	Input and Parameter Validation	25
6.3	Business-State Validation	26
6.4	CORS Assessment	26
6.5	API Response Data Assessment	27
6.5.1	Remediation	27
6.5.2	Before and After Measurement	27
6.6	API Security Test Summary	28
6.7	API Security Conclusion	28
7	Security Findings	29
7.1	Finding Summary	29
7.2	F-01 – API Over-Fetching / Excessive Data Exposure	29
7.2.1	Description	29
7.2.2	Affected Component	30
7.2.3	Evidence	30
7.2.4	Security Impact	30
7.2.5	Remediation	30
7.2.6	Retest Result	31
7.3	F-02 – Missing HTTP Security Headers	31
7.3.1	Description	31
7.3.2	Affected Component	31
7.3.3	Evidence Before Remediation	32
7.3.4	Security Impact	32
7.3.5	Remediation	32
7.3.6	Retest Result	32
7.4	F-03 – Vulnerable Transitive Development Dependencies	33
7.4.1	Description	33
7.4.2	Evidence	33
7.4.3	Risk Assessment	34
7.4.4	Remediation	34
7.4.5	Retest Result	34

7.5	Overall Finding Conclusion	35
8	Security Controls That Passed	36
8.1	Authentication Controls	36
8.2	Authorization and RBAC Controls	36
8.3	Input Validation Controls	37
8.4	CORS Controls	37
8.5	HTTP Security Header Controls	37
8.6	Database Access Controls	38
8.7	Build and Runtime Verification	38
8.8	Security Control Summary	39
9	Remediation and Retesting	40
9.1	Remediation Approach	40
9.2	F-01 Retesting – API Response Data	40
9.3	F-02 Retesting – HTTP Security Headers	41
9.4	F-03 Retesting – Dependency Assessment	41
9.5	Authentication and Authorization Retesting	42
9.6	Build Verification	42
9.7	Runtime Verification	43
9.8	Remediation Summary	43
9.9	Overall Retesting Conclusion	43
10	Risk Summary	44
10.1	Overall Risk Assessment	44
10.2	Risk Rating Summary	44
10.3	F-01 Risk Assessment	44
10.4	F-02 Risk Assessment	45
10.5	F-03 Risk Assessment	45
10.6	Risk Prioritization	45
10.7	Post-Remediation Security Posture	46
11	Security Recommendations	47
11.1	Dependency Management	47
11.2	API Response Data Minimization	47
11.3	Authentication and Authorization	48
11.4	HTTP Security Configuration	48
11.5	Input and Business Logic Validation	48
11.6	Secure Development Practices	49
11.7	Future Security Testing	49

12 Conclusion	50
A Test Evidence	52
A.1 Authentication Testing	52
A.2 Invalid JWT Testing	52
A.3 Valid JWT Testing	53
A.4 Role-Based Authorization Testing	53
A.5 Security Header Testing	53
A.6 CORS Testing	54
B Selected API Test Results	55
B.1 Input Validation	55
B.2 Business-State Validation	55
B.3 API Response Data Minimization	55
B.4 Authentication and Authorization Summary	56
C Dependency Audit Evidence	57
C.1 Initial Dependency Audit	57
C.2 Prisma Security Update	57
C.3 Post-Update Dependency Audit	57
C.4 Dependency Tree Analysis	58
C.5 Forced Remediation Decision	58
D Build and Runtime Verification	59
D.1 Build Verification	59
D.2 Runtime Health Verification	59
D.3 Final Verification Summary	60

1 Executive Summary

SupplyFlow is a full-stack business management and ERP application designed to support business operations including customer management, quotations, sales orders, inventory management, and dispatch workflows. The application consists of a React/TypeScript frontend and a Node.js/Express/TypeScript backend with PostgreSQL as the database and Prisma ORM for database access.

This assessment was conducted as a practical application security and Vulnerability Assessment and Penetration Testing (VAPT) exercise against the locally deployed SupplyFlow backend. The assessment focused on authentication, authorization and role-based access control, API security, input validation, CORS configuration, HTTP security headers, response data exposure, dependency vulnerabilities, and selected source-code security controls.

The assessment identified three principal security findings. The first involved over-fetching of related data from the sales-order API. Although the exposed information was accessible to an authorized Sales User, the API returned broader object fields than were required by the application's business functionality. This was remediated by replacing broad Prisma relation loading with explicit field-level `select` projections. Following remediation, the endpoint continued to function correctly while the observed response size decreased from 9,282 bytes to 7,173 bytes, representing an approximately 22.5% reduction.

The second finding concerned the absence of common HTTP security headers. The application initially exposed the default Express `X-Powered-By` header and did not provide several recommended browser security controls. Helmet was introduced as application middleware to provide a standardized set of HTTP security headers. Retesting confirmed the presence of security headers including Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and related controls, while the Express `X-Powered-By` header was removed.

The third finding concerned high-severity vulnerabilities reported by the project's dependency audit. The initial audit reported seven high-severity findings. Prisma was patched from version 6.19.0 to 6.19.3, after which the audit reported six remaining high-severity findings. Dependency-tree analysis showed that the reported `braces` vulnerability was introduced through the development-only `ts-node-dev` dependency, while

`deepmerge-ts` was introduced through the Prisma CLI/configuration dependency chain. The available automated remediation for the latter would require a breaking dependency change, so a forced upgrade was not applied. The remaining findings were therefore documented and assessed rather than blindly applying a potentially disruptive fix.

Authentication and authorization controls were also validated through controlled API testing. Protected endpoints rejected unauthenticated requests with HTTP 401 responses, invalid JWTs were rejected with HTTP 401, valid authentication credentials successfully issued JWTs, and valid JWTs provided access to authorized endpoints. A Sales User with a valid JWT was correctly prevented from performing an Admin-only inventory modification, receiving HTTP 403.

Following the security modifications, the application successfully passed the production TypeScript build and the backend health endpoint continued to return a successful response. The assessment therefore demonstrates both identification of application security weaknesses and practical remediation and retesting of the identified issues.

2 Application Overview

2.1 SupplyFlow Overview

SupplyFlow is a full-stack business management and Enterprise Resource Planning (ERP) application developed to support core business operations through a centralized web-based system. The application provides functionality for managing customers, enquiries, quotations, sales orders, inventory, and dispatch operations.

The system follows a client-server architecture in which the frontend communicates with a RESTful backend API. The backend is responsible for authentication, authorization, business logic, database operations, and validation of incoming requests.

The security assessment was performed primarily against the backend API because it contains the application's authentication mechanisms, role-based access controls, business workflows, database interactions, and security-sensitive operations.

2.2 Technology Stack

The primary technologies identified during the assessment are:

Component	Technology
Frontend	React / TypeScript
Backend	Node.js / Express / TypeScript
Database	PostgreSQL
ORM	Prisma
Authentication	JSON Web Tokens (JWT)
Password Security	bcrypt
Input Validation	Zod
Security Headers	Helmet
Cross-Origin Control	Express CORS middleware
Testing	cURL / Manual API Testing
Dependency Assessment	npm audit

Table 2.1: SupplyFlow Technology Stack

2.3 Application Architecture

The application uses a layered architecture consisting of the frontend, backend API, middleware, controllers, and PostgreSQL database.

The frontend communicates with backend REST API endpoints over HTTP. Authentication is implemented using JSON Web Tokens. Requests to protected endpoints pass through the authentication middleware, which validates the supplied token and attaches the authenticated user's identity and role to the request.

Authorization is implemented separately through role-based middleware. The application currently distinguishes between the **ADMIN** and **SALES_USER** roles. Administrative operations, such as inventory modification and selected sales-order operations, are restricted to users with the appropriate role.

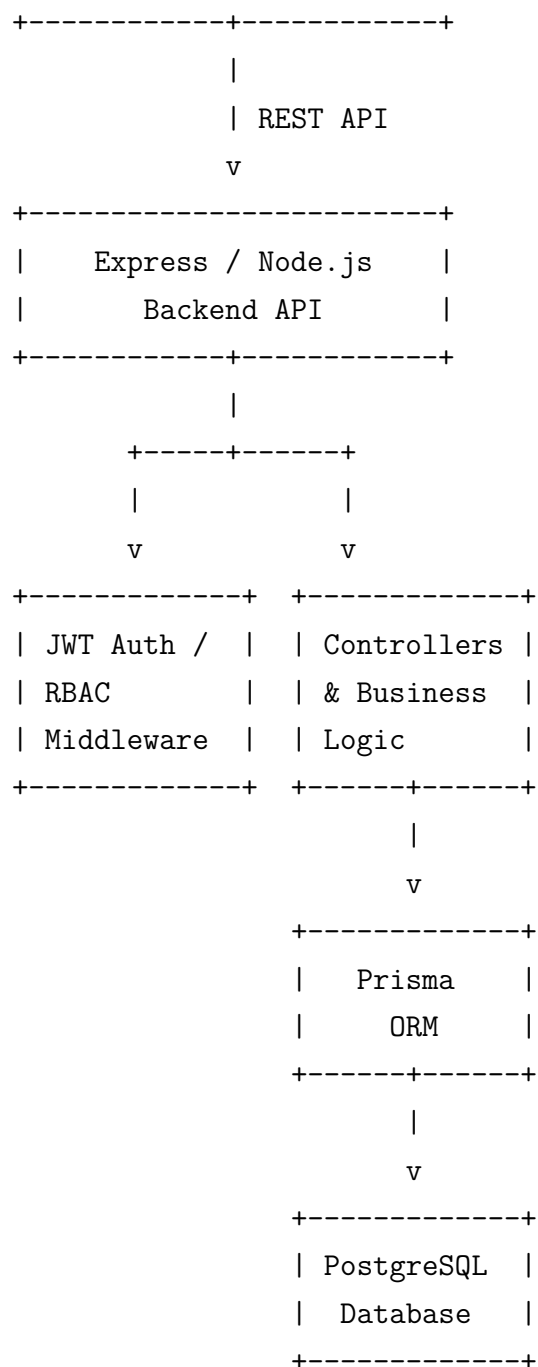
The backend uses Prisma ORM for database access. During the assessment, Prisma queries were also reviewed for response-data minimization. Broad relational data retrieval in the sales-order endpoint was replaced with explicit field-level projections.

A simplified representation of the assessed architecture is:

```

+-----+
|  React / TypeScript  |
|      Frontend        |
+-----+

```



2.4 Security-Relevant Components

The following components were particularly relevant to the security assessment:

- **JWT Authentication:** Used to authenticate users accessing protected API endpoints.
- **Role-Based Authorization:** Used to restrict privileged operations based on the user's assigned role.

- **Input Validation:** Used to validate identifiers, quantities, and business-operation parameters before processing requests.
- **Prisma ORM:** Used for database access and reviewed for secure and minimal data retrieval.
- **Helmet:** Added during remediation to provide HTTP security headers.
- **CORS Configuration:** Reviewed to verify that unauthorized origins were not reflected by the API.
- **Dependency Management:** Assessed using `npm audit` and dependency-tree analysis.

3 Assessment Scope

3.1 Scope

The assessment was conducted against the locally deployed SupplyFlow backend application. The primary objective was to evaluate application-level security controls and identify security weaknesses that could affect the confidentiality, integrity, or availability of the application's business functionality.

The assessment covered the following areas:

- Authentication and JWT token validation.
- Role-Based Access Control (RBAC).
- Authorization of privileged API operations.
- Protected REST API endpoints.
- Input and parameter validation.
- API response data exposure and over-fetching.
- CORS configuration.
- HTTP security headers.
- Dependency vulnerability assessment.
- Relevant backend source-code security controls.
- Security remediation and post-remediation verification.

3.2 In-Scope Components

The following components were considered within the practical scope of the assessment:

Component	Assessment Coverage
Express Backend API	Authentication, authorization, API security
JWT Middleware	Token validation and authentication handling
RBAC Middleware	Role-based access enforcement
Inventory APIs	Authorization and input validation
Sales Order APIs	Authorization and business-state validation
Authentication API	Credential validation and JWT issuance
Prisma Data Access	Query and response-data review
HTTP Configuration	Security headers and CORS
Node.js Dependencies	Vulnerability assessment using npm audit

Table 3.1: In-Scope Components

3.3 Assessment Environment

Testing was performed against a local development instance of the SupplyFlow backend.

Property	Value
Environment	Local development environment
Backend	Node.js / Express / TypeScript
Database	PostgreSQL
Database Access	Prisma ORM
API Port	5000
API Protocol	HTTP (local testing)
Testing Method	Manual API and source-code assessment
Primary Tools	cURL, npm audit, Prisma CLI

Table 3.2: Assessment Environment

3.4 Out of Scope

The assessment was intentionally limited to the application-level controls and local environment described above. The following areas were not subjected to a full security

assessment:

- Production infrastructure and cloud deployment security.
- AWS account-level security configuration.
- Network penetration testing against external infrastructure.
- Denial-of-Service or stress testing.
- Physical security.
- Social engineering or phishing.
- Third-party service security assessments.
- Full frontend penetration testing.
- Database server hardening beyond application-level database access.
- Source-code review of components outside the assessed backend security controls.

3.5 Testing Limitations

The assessment was performed in a controlled local environment using the application's available test accounts and data. Therefore, the results demonstrate the security properties of the assessed application version and configuration but should not be interpreted as a complete security certification of a production deployment.

The dependency findings reported by `npm audit` were also evaluated in the context of the application's dependency tree. Particular attention was given to whether the affected packages were direct runtime dependencies or were introduced through development tooling.

4 Security Testing Methodology

4.1 Testing Approach

The assessment followed a structured application security testing approach combining manual API testing, backend source-code review, dependency analysis, remediation, and post-remediation verification.

Testing was performed against the locally deployed SupplyFlow backend using controlled requests and the application's available test accounts. The objective was to validate security controls from both an unauthenticated and authenticated perspective while avoiding destructive testing against application data.

The assessment process consisted of the following phases:

1. Application and backend architecture review.
2. Identification of authentication and authorization controls.
3. Manual API security testing.
4. Source-code review of security-sensitive middleware and controllers.
5. Identification and validation of security weaknesses.
6. Implementation of security remediations.
7. Dependency vulnerability assessment.
8. Post-remediation testing and build verification.

4.2 Manual API Security Testing

Manual API testing was performed using controlled HTTP requests against the local backend. The tests focused on determining whether protected endpoints correctly enforced authentication, authorization, input validation, and business-state controls.

The following authentication and authorization scenarios were validated:

- Accessing protected endpoints without an authentication token.
- Accessing protected endpoints using an invalid JWT.
- Authenticating with valid credentials and obtaining a JWT.
- Accessing protected resources using a valid JWT.
- Attempting Admin-only operations using a Sales User account.
- Supplying invalid identifiers to protected business operations.
- Attempting invalid business-state transitions.

Expected HTTP status codes were compared with the application's actual responses to determine whether the relevant security control was correctly enforced.

4.3 Source Code Review

Relevant backend source code was manually reviewed to understand the implementation of security controls and to identify potential weaknesses.

The review included:

- JWT authentication middleware.
- Role-based authorization middleware.
- Authentication and login routes.
- Inventory routes and controllers.
- Sales-order routes and controllers.
- Prisma database queries.
- Express application configuration.
- CORS configuration.
- HTTP security middleware.

Particular attention was given to authentication boundaries, authorization checks, parameter validation, database queries, API response structures, and security-related middleware.

4.4 Dependency Vulnerability Assessment

The Node.js dependency tree was assessed using:

```
npm audit
```

The reported vulnerabilities were subsequently examined using:

```
npm ls braces chokidar ts-node-dev deepmerge-ts prisma @prisma/config
```

This dependency-tree analysis was performed to determine how reported vulnerabilities entered the application and whether they originated from direct runtime dependencies or development tooling.

The assessment identified transitive vulnerabilities associated with **braces** and **deepmerge-ts**. Prisma was subsequently updated from version 6.19.0 to version 6.19.3, after which the reported high-severity findings decreased from seven to six.

4.5 Security Header Assessment

The application's HTTP response headers were reviewed before and after introducing the Helmet middleware.

The assessment specifically examined the presence or absence of commonly recommended browser security controls, including:

- Content-Security-Policy.
- X-Content-Type-Options.
- X-Frame-Options.
- Referrer-Policy.
- Cross-Origin-Opener-Policy.
- Cross-Origin-Resource-Policy.
- Strict-Transport-Security.
- Removal of the Express X-Powered-By header.

The results were compared before and after remediation.

4.6 API Response Data Assessment

API response structures were reviewed to determine whether endpoints returned fields beyond those required by their intended business functionality.

The sales-order API was identified as returning broader related-object information than necessary. The original Prisma relation loading was therefore reviewed and replaced with explicit field-level `select` projections.

The endpoint was subsequently retested to verify that required business information remained available while unnecessary metadata was removed.

4.7 Remediation and Retesting

Security findings were not considered closed immediately after code modification. Each remediation was followed by verification to determine whether the intended security control was restored without breaking the application's functionality.

Retesting included:

- Re-running affected API requests.
- Verifying expected HTTP status codes.
- Confirming required API functionality remained available.
- Reviewing security response headers after Helmet implementation.
- Re-running dependency vulnerability assessment after the Prisma update.
- Running the TypeScript production build.
- Verifying the backend health endpoint after the changes.

4.8 Tools Used

Tool / Technology	Purpose
cURL	Manual HTTP and API security testing
Node.js	Automated local API test execution
npm audit	Dependency vulnerability assessment
npm ls	Dependency-tree analysis
Prisma CLI	Database client and ORM verification
TypeScript Compiler	Post-remediation build verification
Express	Backend application under assessment
Helmet	HTTP security-header remediation
VS Code / Source Review	Manual backend code analysis

Table 4.1: Security Assessment Tools

5 Authentication and Authorization Assessment

5.1 JWT Authentication

SupplyFlow uses JSON Web Tokens (JWT) to authenticate users accessing protected API resources. The authentication middleware checks the **Authorization** HTTP header for a Bearer token and validates the token before allowing the request to continue.

Requests without a valid authentication token are rejected before reaching the protected business logic.

The authentication flow was assessed using both negative and positive test cases.

5.2 Authentication Test Results

The following tests were performed against the protected inventory API.

ID	Test Case	Expected	Result
AUTH-01	Access protected endpoint without JWT	401 Unauthorized	PASS
AUTH-02	Access protected endpoint using invalid JWT	401 Unauthorized	PASS
AUTH-03	Authenticate using valid credentials	Successful authentication / JWT issued	PASS
AUTH-04	Access protected endpoint using valid JWT	200 OK	PASS

Table 5.1: JWT Authentication Test Results

5.2.1 Unauthenticated Request

A request was sent to the protected inventory endpoint without an `Authorization` header:

```
GET /api/inventory
```

The application returned:

```
HTTP/1.1 401 Unauthorized
```

```
{"success":false,"message":"Authentication required"}
```

This confirms that the endpoint does not permit anonymous access.

5.2.2 Invalid JWT

A request containing an intentionally invalid JWT was then submitted:

```
Authorization: Bearer fake.invalid.token
```

The application returned:

```
HTTP/1.1 401 Unauthorized
```

```
{"success":false,"message":"Invalid or expired token"}
```

The result confirms that an arbitrary token cannot be used to bypass authentication.

5.2.3 Valid Authentication

Valid application credentials were used to authenticate against the login endpoint. The application successfully issued a JWT.

The token was subsequently used to access the protected inventory endpoint. The authenticated request returned:

```
HTTP 200
```

with the expected inventory data.

This confirms the complete authentication flow:

Valid Credentials

|

v

JWT Issued

|

v

JWT Validation

|

v

Protected API Access

|

v

HTTP 200

5.3 Role-Based Access Control

Authentication alone does not determine whether a user can perform a particular operation. SupplyFlow additionally implements role-based authorization using the authenticated user's role.

The assessed roles include:

- ADMIN
- SALES_USER

The authorization middleware checks whether the authenticated user's role is included in the roles permitted to perform the requested operation.

Administrative inventory modification is restricted to the ADMIN role.

5.4 Authorization Test Results

A valid Sales User JWT was used to attempt an inventory modification that requires administrative privileges.

ID	Test Case	Expected	Result
AUTHZ-01	Sales User attempts Admin-only inventory modification	403 Forbidden	PASS
AUTHZ-02	Sales User attempts Admin-only sales-order operation	403 Forbidden	PASS

Table 5.2: Authorization Test Results

5.4.1 Admin-Only Inventory Modification

A Sales User with a valid JWT attempted to modify inventory through the protected inventory endpoint.

The application returned:

HTTP 403

```
{"success":false,
  "message":"You are not authorized to perform this operation"}
```

The request was therefore rejected despite the user being fully authenticated.

This demonstrates correct separation between authentication and authorization:

Authenticated User

|

v

Role = SALES_USER

|

v

Admin-only Operation

|

v

HTTP 403 Forbidden

5.4.2 Admin-Only Sales Order Operations

The assessment also verified that administrative sales-order operations were protected by role-based authorization. A Sales User attempting a restricted operation received an HTTP 403 response.

This confirms that privileged operations are protected at the route/middleware level rather than relying solely on frontend restrictions.

5.5 Authentication and Authorization Conclusion

The assessed authentication and authorization controls operated as expected during testing. Protected endpoints rejected unauthenticated and invalid-token requests, valid authentication credentials resulted in successful JWT issuance, and valid JWTs provided access to authorized resources.

Role-based authorization also correctly prevented a Sales User from performing Admin-only operations.

No authentication or authorization bypass was identified during the tested scenarios.

6 API Security Assessment

6.1 API Security Overview

The SupplyFlow backend exposes RESTful API endpoints for authentication, inventory, sales-order processing, and related business operations. The API security assessment focused on request validation, authorization enforcement, cross-origin configuration, business-state validation, and the amount of information returned by API responses.

The assessment combined manual HTTP testing with source-code review of the Express routes, middleware, controllers, and Prisma queries.

6.2 Input and Parameter Validation

Input validation was reviewed on security-sensitive sales-order and inventory operations. The application validates identifiers and business parameters before performing database operations.

For example, sales-order operations validate that the supplied sales-order identifier is a positive integer.

The following invalid identifier scenarios were tested:

ID	Test Case	Expected	Result
API-01	Non-numeric Sales Order ID	400 Bad Request	PASS
API-02	Negative Sales Order ID	400 Bad Request	PASS
API-03	Non-existent Sales Order ID	Appropriate error response	PASS

Table 6.1: API Parameter Validation Results

For example, an Admin request using the value `abc` as the sales-order identifier returned:

HTTP 400

```
{"success":false,"message":"Invalid Sales Order ID"}
```

A negative identifier was similarly rejected.

These controls reduce the risk of malformed identifiers reaching business logic or database operations.

6.3 Business-State Validation

The sales-order workflow implements state-based controls to prevent invalid business operations.

For example, a sales order can only be confirmed when it is in the expected **PENDING** state. During testing, an attempt was made to confirm an order that was already in the **CONFIRMED** state.

The application returned:

HTTP 400

```
{"success":false,  
  "message":"Only a PENDING Sales Order can be confirmed"}
```

This demonstrates that authorization alone is not relied upon to protect sensitive business operations. The backend also validates whether the requested state transition is logically permitted.

6.4 CORS Assessment

The Cross-Origin Resource Sharing (CORS) configuration was assessed by sending a request containing an unauthorized origin.

The test used:

Origin: `http://evil.example`

The backend did not reflect the unauthorized origin. Instead, the response contained the configured frontend origin:

Access-Control-Allow-Origin: `http://localhost:5173`
Vary: Origin

The protected request itself was also rejected because authentication was not provided.

ID	Test Case	Expected	Result
API-04	Request with unauthorized Origin header	Unauthorized origin not reflected	PASS

Table 6.2: CORS Assessment

No evidence was observed during the tested scenario that the application reflected the unauthorized origin.

6.5 API Response Data Assessment

The sales-order endpoint was reviewed for unnecessary information disclosure through API responses.

The original implementation used broad Prisma relation loading, including related customer, quotation, enquiry, product, and dispatch objects. Although the Sales User was intentionally authorized to access the sales-order information, the response contained additional metadata that was not required by the assessed business use case.

This was classified as an API data-minimization issue rather than an authorization bypass.

6.5.1 Remediation

The broad relation loading was replaced with explicit Prisma `select` projections. The revised query specifies the exact fields required from the sales order and its related entities.

This approach limits the API response to the required business information and reduces the likelihood of unintentionally exposing additional database fields when the schema changes in the future.

6.5.2 Before and After Measurement

The endpoint was tested before and after the remediation.

Measurement	Response Size
Before remediation	9,282 bytes
After remediation	7,173 bytes
Reduction	Approximately 22.5%

Table 6.3: Sales Order API Response Data Reduction

Following the change, the endpoint continued to return HTTP 200 and retained the business information required by the Sales User workflow.

6.6 API Security Test Summary

ID	Control Tested	Result
API-01	Sales-order identifier validation	PASS
API-02	Negative identifier rejection	PASS
API-03	Non-existent resource handling	PASS
API-04	CORS origin handling	PASS
API-05	Business-state validation	PASS
API-06	API response data minimization	REMEDIATED

Table 6.4: API Security Assessment Summary

6.7 API Security Conclusion

The tested API controls successfully enforced parameter validation, business-state validation, role-based access restrictions, and the assessed CORS policy.

One data-minimization weakness was identified in the sales-order response. The issue was remediated through explicit Prisma field selection and successfully retested without breaking the endpoint's intended business functionality.

7 Security Findings

7.1 Finding Summary

The assessment identified three principal security findings. Two findings were remediated during the assessment, while the remaining dependency-related finding was assessed and mitigated through dependency analysis and a safe version update.

ID	Finding	Severity	Status
F-01	API Over-Fetching / Excessive Data Exposure	Medium	Remediated
F-02	Missing HTTP Security Headers	Medium	Remediated
F-03	Vulnerable Transitive Development Dependencies	High	Assessed / Mitigated

Table 7.1: Security Findings Summary

7.2 F-01 – API Over-Fetching / Excessive Data Exposure

Severity: Medium

Status: Remediated

7.2.1 Description

The sales-order API initially used broad Prisma relation loading when retrieving sales orders and their related entities. This caused the API to return a wider set of fields from related customer, quotation, enquiry, product, and dispatch records than was necessary for the intended business functionality.

The issue did not represent an authorization bypass because Sales Users were intentionally permitted to access sales-order information. Instead, it represented a data-minimization and API response-control weakness.

Returning unnecessary fields increases the amount of information exposed by an API and can increase the impact of future schema changes if newly added database fields are automatically included in API responses.

7.2.2 Affected Component

GET /api/sales-orders

The affected implementation was the Prisma query used by the `getSalesOrders` controller.

7.2.3 Evidence

The original implementation used broad relational inclusion for associated records. The API response was measured at approximately:

9,282 bytes

The endpoint was reviewed to identify fields that were not required by the assessed business workflow.

7.2.4 Security Impact

Excessive API responses can unnecessarily expose database attributes to authenticated clients. While the information was available to an authorized Sales User in this application, unnecessary response fields increase the application's information exposure surface and make future accidental disclosure more likely.

7.2.5 Remediation

The broad Prisma relation loading was replaced with explicit field-level `select` projections.

The revised implementation explicitly defines the fields returned from:

- Sales orders.
- Customers.
- Quotations.

- Enquiries.
- Sales-order items.
- Products.
- Dispatch records.

This implements a least-data principle at the API response layer.

7.2.6 Retest Result

The endpoint was retested after remediation.

The resulting response size was:

7,173 bytes

This represents an approximate reduction of:

22.5\%

The endpoint continued to return HTTP 200 and retained the business information required by the application.

Final Status: FIXED AND RETESTED

7.3 F-02 – Missing HTTP Security Headers

Severity: Medium

Status: Remediated

7.3.1 Description

During the initial HTTP response-header assessment, the Express backend did not provide a comprehensive set of commonly recommended browser security headers.

The application exposed the default Express **X-Powered-By** header and did not initially provide controls such as Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Referrer-Policy.

7.3.2 Affected Component

The affected component was the global Express application configuration.

7.3.3 Evidence Before Remediation

The initial HTTP response from the application did not contain the expected set of security headers. The `Express X-Powered-By` header was also present.

This increased the application's exposure to browser-side security risks that can be reduced through appropriate HTTP response headers.

7.3.4 Security Impact

Missing browser security headers can reduce the browser's ability to enforce defensive security policies. Depending on the application's deployment and attack surface, this can increase exposure to risks such as clickjacking, MIME-type confusion, information disclosure, and certain classes of browser-based attacks.

The finding was therefore treated as a security-hardening weakness rather than evidence of a confirmed exploit.

7.3.5 Remediation

The Helmet middleware was installed and added globally to the Express application.

The application was updated to include:

```
app.use(helmet());
```

This enabled a standardized set of HTTP security headers.

7.3.6 Retest Result

After remediation, the application's HTTP responses were verified to include security headers such as:

- Content-Security-Policy.
- Cross-Origin-Opener-Policy.
- Cross-Origin-Resource-Policy.
- Referrer-Policy.
- Strict-Transport-Security.
- X-Content-Type-Options.
- X-Frame-Options.
- X-DNS-Prefetch-Control.

The Express **X-Powered-By** header was no longer present.

The application continued to operate successfully after the change.

Note: Strict-Transport-Security is intended for HTTPS deployments. Its presence in the local test response was recorded as part of the Helmet configuration; production deployment should use HSTS only with an appropriately configured HTTPS environment.

Final Status: FIXED AND RETESTED

7.4 F-03 – Vulnerable Transitive Development Dependencies

Severity: High

Status: Assessed / Mitigated

7.4.1 Description

The initial Node.js dependency audit reported seven high-severity vulnerabilities. Further investigation showed that the reported vulnerabilities were primarily transitive dependencies rather than direct application dependencies.

The dependency tree was analyzed to determine the source and practical context of the reported vulnerabilities.

7.4.2 Evidence

The dependency audit identified a high-severity vulnerability affecting **braces**. The dependency chain was:

```
ts-node-dev
|
+-- chokidar@3.6.0
|
+-- braces@3.0.3
```

The project declares **ts-node-dev** under **devDependencies**, and the production application is started using the compiled JavaScript application:

```
node dist/server.js
```

A second high-severity vulnerability involved **deepmerge-ts**. The dependency chain was identified as:

```
prisma@6.19.3
|
+-- @prisma/config@6.19.3
|
+-- deepmerge-ts@7.1.5
```

The Prisma CLI is also declared under `devDependencies`.

7.4.3 Risk Assessment

The reported vulnerabilities require contextual assessment because the affected packages are introduced through development tooling rather than being direct business runtime dependencies.

The **braces** vulnerability has no automated fix available according to the audit output.

The available automated remediation for **deepmerge-ts** required a breaking dependency change that would have installed an older Prisma version. Applying a forced dependency update without validating application compatibility was therefore considered an inappropriate remediation strategy.

7.4.4 Remediation

The Prisma packages were safely patched from:

```
Prisma 6.19.0
|
v
Prisma 6.19.3
```

Following this update, the number of high-severity findings reported by `npm audit` decreased from seven to six.

The dependency tree was subsequently reviewed to understand the remaining findings.

The project was not subjected to:

```
npm audit fix --force
```

because the available forced remediation would introduce a breaking Prisma dependency change.

7.4.5 Retest Result

After the Prisma update:

- Prisma Client generation was completed after resolving the local Windows file-lock condition.
- The application successfully passed the TypeScript build.
- The backend health endpoint continued to return a successful response.
- The dependency tree was re-evaluated.
- The audit count decreased from seven to six high-severity findings.

The remaining findings were documented rather than blindly force-fixed.

Final Status: ASSESSED AND MITIGATED

7.5 Overall Finding Conclusion

The assessment resulted in two findings that were directly remediated in the application and one dependency-related finding that was investigated, partially mitigated, and documented according to its dependency context.

The remediation process was followed by application build and runtime verification to confirm that the security improvements did not break the assessed backend functionality.

8 Security Controls That Passed

The assessment identified several security controls that operated as intended during testing. These controls were validated through controlled API requests, source-code review, configuration review, and post-remediation testing.

8.1 Authentication Controls

The JWT authentication mechanism successfully enforced authentication on protected endpoints.

The following controls were verified:

- Requests without an authentication token were rejected with HTTP 401.
- Requests containing an invalid JWT were rejected with HTTP 401.
- Valid credentials successfully resulted in JWT issuance.
- A valid JWT successfully authenticated requests to protected endpoints.

No authentication bypass was identified within the tested scenarios.

8.2 Authorization and RBAC Controls

Role-based authorization was successfully enforced for privileged operations.

The application distinguishes between the `ADMIN` and `SALES_USER` roles. Administrative operations were protected by explicit authorization middleware.

A Sales User possessing a valid JWT was unable to perform an Admin-only inventory modification and received HTTP 403 Forbidden.

The assessment also confirmed that selected Admin-only sales-order operations rejected Sales User requests.

These results demonstrate that authorization is enforced on the backend rather than being dependent solely on frontend controls.

8.3 Input Validation Controls

The application performed validation of security-sensitive request parameters.

The assessment verified rejection of:

- Non-numeric sales-order identifiers.
- Negative sales-order identifiers.
- Invalid business-state transitions.

For example, an attempt to confirm a Sales Order that was already in the **CONFIRMED** state was rejected with HTTP 400.

This demonstrates that the backend validates business rules before performing sensitive operations.

8.4 CORS Controls

The CORS configuration was tested using an unauthorized origin.

The test supplied:

Origin: `http://evil.example`

The application did not reflect this origin in the **Access-Control-Allow-Origin** response header. Instead, the configured frontend origin was returned.

The tested CORS control therefore passed the assessment scenario.

8.5 HTTP Security Header Controls

The initial assessment identified missing HTTP security headers. Following remediation, Helmet was introduced as a global Express middleware.

Post-remediation testing confirmed the presence of multiple browser security headers, including:

- Content-Security-Policy.
- X-Content-Type-Options.
- X-Frame-Options.
- Referrer-Policy.
- Cross-Origin-Opener-Policy.

- Cross-Origin-Resource-Policy.

The default Express **X-Powered-By** response header was also removed.

The security-header control therefore passed the post-remediation verification.

8.6 Database Access Controls

The assessed backend uses Prisma ORM for database interaction. Relevant controllers were reviewed to verify that sensitive operations were performed through controlled application logic.

Security-sensitive operations such as inventory updates and sales-order state changes include validation and authorization checks before database modifications are performed.

The assessed implementation also uses parameterized database operations for relevant inventory calculations and state-changing workflows, reducing exposure to direct SQL injection through the reviewed operations.

8.7 Build and Runtime Verification

Following the security modifications, the application was successfully compiled using the TypeScript build process.

The backend health endpoint was subsequently verified:

```
GET /api/health
```

The application returned:

```
{"success":true,"message":"SupplyFlow ERP API is running"}
```

This confirmed that the security changes and dependency update did not prevent the backend from starting or serving API requests.

8.8 Security Control Summary

Control	Validation	Result
JWT Authentication	Missing and invalid tokens rejected	PASS
JWT Issuance	Valid credentials produced JWT	PASS
JWT Protected Access	Valid JWT granted authorized access	PASS
RBAC	Sales User blocked from Admin operation	PASS
Input Validation	Invalid identifiers rejected	PASS
Business-State Validation	Invalid state transition rejected	PASS
CORS	Unauthorized origin not reflected	PASS
HTTP Security Headers	Helmet implemented and verified	PASS
API Data Minimization	Excessive fields removed	REMEDIATED
Dependency Assessment	Dependency tree reviewed	ASSESSED
Build Verification	TypeScript build completed successfully	PASS
Runtime Verification	Health endpoint returned success	PASS

Table 8.1: Security Control Assessment Summary

9 Remediation and Retesting

9.1 Remediation Approach

The identified findings were addressed using a risk-based remediation approach. Changes were implemented directly in the SupplyFlow backend and subsequently verified through repeat testing.

The remediation process focused on improving security without unnecessarily changing existing business functionality.

The following remediation activities were performed:

1. Reduced unnecessary API response data through explicit Prisma field projections.
2. Added Helmet middleware for HTTP security hardening.
3. Updated Prisma from version 6.19.0 to version 6.19.3.
4. Re-assessed the remaining dependency vulnerabilities and their dependency paths.
5. Rebuilt the application after the security modifications.
6. Verified that the backend continued to operate after remediation.

9.2 F-01 Retesting – API Response Data

The sales-order API was retested following the introduction of explicit Prisma `select` projections.

Metric	Result
Original response size	9,282 bytes
Remediated response size	7,173 bytes
Reduction	Approximately 22.5%
Endpoint status after remediation	HTTP 200
Required business data	Retained

Table 9.1: F-01 Remediation Verification

The endpoint continued to provide the customer, sales-order, quotation, product, and dispatch information required by the application's intended Sales User workflow.

The remediation therefore reduced unnecessary response data without breaking the endpoint.

Retest Status: PASS

9.3 F-02 Retesting – HTTP Security Headers

The application was retested after introducing Helmet as global Express middleware.

The following response headers were observed after remediation:

```
Content-Security-Policy
Cross-Origin-Opener-Policy
Cross-Origin-Resource-Policy
Referrer-Policy
Strict-Transport-Security
X-Content-Type-Options
X-Frame-Options
X-DNS-Prefetch-Control
```

The Express `X-Powered-By` header was also no longer present.

The application continued to respond normally after the middleware was introduced.

Retest Status: PASS

9.4 F-03 Retesting – Dependency Assessment

The dependency assessment was repeated after updating Prisma.

The initial dependency audit reported:

7 high severity vulnerabilities

Prisma was updated from version 6.19.0 to version 6.19.3.

The subsequent audit reported:

6 high severity vulnerabilities

The dependency tree was then examined using:

```
npm ls braces chokidar ts-node-dev deepmerge-ts prisma @prisma/config
```

The analysis confirmed that the remaining reported vulnerabilities were introduced through transitive dependencies of development tooling.

The available forced remediation was not applied because it would introduce a breaking Prisma dependency change.

Retest Status: ASSESSED / MITIGATED

9.5 Authentication and Authorization Retesting

Authentication and authorization controls were revalidated after the application changes.

The following results were confirmed:

Test	Result
Unauthenticated request to protected API	HTTP 401
Invalid JWT request	HTTP 401
Valid JWT request	HTTP 200
Sales User attempting Admin operation	HTTP 403

Table 9.2: Post-Remediation Authentication and Authorization Verification

No regression was observed in the tested authentication or authorization controls.

9.6 Build Verification

After the security modifications and Prisma update, the backend was compiled using:

```
npm run build
```

The build completed successfully without TypeScript compilation errors.

This provided an additional verification that the dependency update and security modifications remained compatible with the application's source code.

9.7 Runtime Verification

The backend was started after the remediation and tested using the application health endpoint:

GET /api/health

The endpoint returned:

```
{"success":true,"message":"SupplyFlow ERP API is running"}
```

This confirmed that the backend remained operational after the security changes.

9.8 Remediation Summary

ID	Finding	Remediation	Retest
F-01	API over-fetching	Explicit Prisma field projections	PASS
F-02	Missing HTTP security headers	Helmet middleware	PASS
F-03	Transitive dependency vulnerabilities	Prisma patch update and dependency-tree assessment	ASSESSED

Table 9.3: Remediation and Retesting Summary

9.9 Overall Retesting Conclusion

The post-remediation assessment confirmed that the implemented security improvements were effective for the tested scenarios.

The API response-data issue was successfully remediated, HTTP security headers were successfully introduced and verified, and the dependency findings were investigated and assessed according to their dependency context.

The application successfully passed both build and runtime verification following the security changes.

10 Risk Summary

10.1 Overall Risk Assessment

The assessment identified three principal security findings across the evaluated application components. Two findings were directly remediated and successfully retested. The remaining dependency-related finding was investigated through dependency-tree analysis and partially mitigated through a safe Prisma patch update.

The findings should be interpreted in the context of the local assessment environment and the specific application version evaluated.

10.2 Risk Rating Summary

ID	Finding	Severity	Status	Residual Risk
F-01	API Over-Fetching / Excessive Data Exposure	Medium	Remediated	Low
F-02	Missing HTTP Security Headers	Medium	Remediated	Low
F-03	Vulnerable Transitive Development Dependencies	High	Assessed / Mitigated	Medium

Table 10.1: Overall Risk Summary

10.3 F-01 Risk Assessment

The API over-fetching issue was classified as Medium because the endpoint returned additional information to an already authorized user rather than allowing an unauthorized

party to access protected resources.

The issue was remediated by explicitly selecting the fields required by the business workflow. Post-remediation testing confirmed that the endpoint continued to function correctly.

The residual risk is considered Low provided that explicit response projections are maintained as the database schema evolves.

10.4 F-02 Risk Assessment

The missing HTTP security headers were classified as Medium because they represented a security-hardening weakness rather than a confirmed direct compromise.

The application was updated to use Helmet, and the resulting HTTP responses were verified to contain multiple recommended security headers.

The residual risk is considered Low for the assessed configuration, subject to correct HTTPS configuration and appropriate production deployment practices.

10.5 F-03 Risk Assessment

The dependency vulnerability finding was initially classified as High based on the severity reported by `npm audit`. However, subsequent dependency-tree analysis provided important context regarding practical exposure.

The identified `braces` vulnerability originates through the development-only `ts-node-dev` dependency chain. The `deepmerge-ts` vulnerability originates through the Prisma CLI/configuration dependency chain.

The project was patched from Prisma 6.19.0 to 6.19.3, reducing the number of reported high-severity findings from seven to six. The remaining findings were not force-fixed because the available automated remediation would introduce a breaking dependency change.

The residual risk is therefore considered Medium and should remain subject to dependency monitoring and future remediation when compatible fixes become available.

10.6 Risk Prioritization

Based on the assessment results, the recommended remediation priority is:

1. **F-03 – Dependency Vulnerabilities:** Continue monitoring and remediate when a compatible non-breaking fix becomes available.
2. **F-01 – API Over-Fetching:** Maintain explicit response projections and review API responses when database schemas change.

3. **F-02 – HTTP Security Headers:** Maintain Helmet configuration and ensure HTTPS is correctly configured in production.

10.7 Post-Remediation Security Posture

Following the implemented remediation activities, the assessed application demonstrated effective controls for:

- JWT authentication.
- Invalid-token rejection.
- Role-based authorization.
- Privileged operation protection.
- API parameter validation.
- Business-state validation.
- CORS origin handling.
- HTTP security-header enforcement.
- API response data minimization.

The application also successfully passed build and runtime verification after the security changes.

The assessment therefore indicates a generally controlled application security posture for the tested scenarios, with dependency management remaining the principal area requiring continued attention.

11 Security Recommendations

The following recommendations are based on the findings identified during the assessment, the security controls reviewed, and the post-remediation results.

11.1 Dependency Management

The project should maintain a regular dependency monitoring and vulnerability management process.

Recommended actions include:

- Regularly execute `npm audit` as part of dependency maintenance.
- Review transitive dependencies when high-severity vulnerabilities are reported.
- Prefer compatible, non-breaking security updates over forced dependency changes.
- Monitor the `ts-node-dev`, Prisma, and related dependency chains for future security fixes.
- Periodically review whether development dependencies are included in production build artifacts or deployment environments.

11.2 API Response Data Minimization

The explicit Prisma field-selection approach introduced during remediation should be maintained across API endpoints.

Developers should avoid broad relational loading when an endpoint only requires a subset of fields. New API endpoints should explicitly define the fields exposed to clients.

This reduces unnecessary information exposure and helps prevent future database schema changes from unintentionally expanding API responses.

11.3 Authentication and Authorization

The existing JWT authentication and role-based authorization controls should be maintained and reviewed whenever new API endpoints are introduced.

Recommended practices include:

- Require authentication on all protected resources.
- Apply authorization checks at the backend API layer.
- Avoid relying exclusively on frontend role restrictions.
- Review privileged endpoints whenever new business operations are added.
- Continue testing both authenticated and unauthorized access scenarios.

11.4 HTTP Security Configuration

Helmet should remain enabled as part of the global Express security configuration.

For production deployment, the HTTP security configuration should additionally be reviewed against the actual deployment architecture.

In particular:

- Production traffic should use HTTPS.
- HSTS should only be enabled when HTTPS is correctly configured.
- Content-Security-Policy should be reviewed against the final frontend requirements.
- Security headers should be periodically verified after framework or middleware updates.

11.5 Input and Business Logic Validation

Backend validation should continue to be applied to all security-sensitive parameters and business operations.

Recommended controls include:

- Validate identifiers before database operations.
- Reject invalid or unexpected input types.
- Validate numeric ranges and business constraints.
- Enforce valid state transitions on workflow operations.
- Perform authorization checks before state-changing operations.

11.6 Secure Development Practices

Security testing should be incorporated into the normal development lifecycle.

Recommended practices include:

- Perform source-code security reviews for security-sensitive changes.
- Add automated tests for authentication and authorization boundaries.
- Include dependency scanning in the development workflow.
- Review API responses whenever new database fields are introduced.
- Perform regression testing after security-related dependency updates.

11.7 Future Security Testing

The current assessment focused on application-level API security in a controlled local environment. A future assessment of a production-like deployment could expand coverage to include:

- Comprehensive API endpoint enumeration and testing.
- Object-level authorization testing across all resources.
- SQL injection testing across all user-controlled inputs.
- File-upload security testing where applicable.
- Session and token lifecycle testing.
- Rate-limiting and abuse-control testing.
- Frontend security assessment.
- HTTPS and TLS configuration assessment.
- Production infrastructure and cloud security assessment.

These areas were outside the scope of the current assessment and should not be interpreted as vulnerabilities identified during this engagement.

12 Conclusion

This assessment evaluated the application-level security posture of the SupplyFlow ERP backend through a combination of manual API testing, source-code review, dependency analysis, security configuration review, remediation, and post-remediation testing.

The assessment identified three principal findings. The first finding involved unnecessary API response data in the sales-order endpoint. This was remediated by replacing broad Prisma relation loading with explicit field-level projections. The remediated endpoint continued to provide the required business functionality while reducing the measured response size by approximately 22.5%.

The second finding involved missing HTTP security headers. Helmet was introduced into the Express application and subsequent testing confirmed the presence of multiple security headers while removing the default Express **X-Powered-By** header.

The third finding involved high-severity transitive dependency vulnerabilities. Dependency-tree analysis established that the identified packages were introduced through development tooling and Prisma configuration dependencies. Prisma was safely updated from version 6.19.0 to version 6.19.3, reducing the reported high-severity findings from seven to six. The remaining findings were documented rather than force-fixed because the available automated remediation would introduce a breaking dependency change.

The assessment also validated the application's core authentication and authorization controls. Unauthenticated requests and invalid JWTs were rejected with HTTP 401, valid authentication successfully granted access to protected resources, and a Sales User was prevented from performing Admin-only operations with HTTP 403.

Following remediation, the application successfully passed the TypeScript build and continued to operate through its health endpoint. These results demonstrate that the security improvements could be implemented without disrupting the tested business functionality.

The assessment was conducted in a controlled local environment and therefore does not constitute a complete production penetration test or security certification. However, it demonstrates a practical application-security workflow covering vulnerability identification, risk assessment, remediation, and retesting.

Overall, the assessment established a stronger security baseline for the evaluated SupplyFlow backend while identifying dependency management as an area requiring contin-

ued monitoring and future remediation.

A Test Evidence

This appendix presents the principal security testing evidence collected during the SupplyFlow application security assessment.

A.1 Authentication Testing

The protected inventory endpoint was tested without an authentication token.

```
GET /api/inventory
```

Observed result:

```
HTTP/1.1 401 Unauthorized
```

```
{"success":false,"message":"Authentication required"}
```

This confirmed that protected API resources cannot be accessed anonymously.

A.2 Invalid JWT Testing

An intentionally invalid JWT was supplied to the protected inventory endpoint.

```
Authorization: Bearer fake.invalid.token
```

Observed result:

```
HTTP/1.1 401 Unauthorized
```

```
{"success":false,"message":"Invalid or expired token"}
```

This confirmed that an invalid token does not bypass JWT authentication.

A.3 Valid JWT Testing

Valid application credentials were used to authenticate successfully and obtain a JWT. The JWT was then used to access the protected inventory endpoint.

Observed result:

HTTP 200

The API returned the expected inventory information.

A.4 Role-Based Authorization Testing

A valid Sales User JWT was used to attempt an Admin-only inventory modification.

Observed result:

HTTP 403

```
{"success":false,  
  "message":"You are not authorized to perform this operation"}
```

This confirmed that a valid authenticated identity does not automatically provide privileged authorization.

A.5 Security Header Testing

Before remediation, the Express application did not provide the complete set of recommended HTTP security headers and exposed the default Express **X-Powered-By** header.

After introducing Helmet, the following headers were observed:

```
Content-Security-Policy  
Cross-Origin-Opener-Policy  
Cross-Origin-Resource-Policy  
Origin-Agent-Cluster  
Referrer-Policy  
Strict-Transport-Security  
X-Content-Type-Options  
X-DNS-Prefetch-Control  
X-Download-Options  
X-Frame-Options  
X-Permitted-Cross-Domain-Policies  
X-XSS-Protection
```

The **X-Powered-By** header was no longer present.

A.6 CORS Testing

A request containing an unauthorized origin was tested:

Origin: `http://evil.example`

The response did not reflect the unauthorized origin. The observed configuration returned:

Access-Control-Allow-Origin: `http://localhost:5173`

Vary: Origin

The tested CORS control therefore passed.

B Selected API Test Results

B.1 Input Validation

Invalid Sales Order identifiers were tested against the sales-order operations.

A non-numeric identifier produced:

HTTP 400

```
{"success":false,"message":"Invalid Sales Order ID"}
```

A negative identifier was also rejected with HTTP 400.

These results confirmed that the backend validates Sales Order identifiers before processing the requested operation.

B.2 Business-State Validation

An attempt was made to confirm a Sales Order that was already in the CONFIRMED state.

The backend returned:

HTTP 400

```
{"success":false,  
  "message":"Only a PENDING Sales Order can be confirmed"}
```

This confirmed enforcement of the expected business-state transition.

B.3 API Response Data Minimization

The sales-order API was assessed before and after remediation.

Measurement	Value
Original response size	9,282 bytes
Remediated response size	7,173 bytes
Reduction	Approximately 22.5%
Post-remediation HTTP status	200 OK

Table B.1: API Response Data Minimization Evidence

The remediation replaced broad Prisma relation loading with explicit field-level `select` projections.

The endpoint continued to return the business information required by the application after the change.

B.4 Authentication and Authorization Summary

Test	HTTP Status	Result
No JWT	401	Authentication rejected
Invalid JWT	401	Invalid token rejected
Valid JWT	200	Authorized access granted
Sales User / Admin operation	403	Privileged operation rejected
Invalid Sales Order ID	400	Input rejected
Invalid business-state transition	400	Operation rejected

Table B.2: Selected API Security Test Results

C Dependency Audit Evidence

C.1 Initial Dependency Audit

The initial execution of:

```
npm audit
```

reported:

```
7 high severity vulnerabilities
```

The reported findings included vulnerabilities associated with transitive dependencies such as `braces` and `deepmerge-ts`.

C.2 Prisma Security Update

The Prisma packages were updated from version 6.19.0 to version 6.19.3:

```
@prisma/client 6.19.3
prisma          6.19.3
```

Following the update, the application was regenerated and successfully built.

C.3 Post-Update Dependency Audit

The subsequent execution of:

```
npm audit
```

reported:

```
6 high severity vulnerabilities
```

Therefore, the Prisma update reduced the reported high-severity findings from seven to six.

C.4 Dependency Tree Analysis

The following command was used to identify the dependency paths:

```
npm ls braces chokidar ts-node-dev deepmerge-ts prisma @prisma/config
```

The relevant dependency relationships were:

```
backend@1.0.0
|
+-- @prisma/client@6.19.3
|   '-- prisma@6.19.3 deduped
|
+-- prisma@6.19.3
|   '-- @prisma/config@6.19.3
|       +-- c12@3.1.0
|           |   '-- chokidar@4.0.3
|               '-- deepmerge-ts@7.1.5
|
'-- ts-node-dev@2.0.0
    '-- chokidar@3.6.0
        '-- braces@3.0.3
```

The analysis confirmed that `ts-node-dev` is declared under `devDependencies`. The Prisma CLI is also declared under `devDependencies`.

The remaining findings were therefore assessed in the context of the application's dependency structure rather than being blindly remediated using a forced dependency change.

C.5 Forced Remediation Decision

The available automated remediation for the `deepmerge-ts` finding required a breaking Prisma version change.

The command:

```
npm audit fix --force
```

was therefore intentionally not executed.

This decision was based on the need to avoid introducing an unvalidated breaking dependency change merely to reduce the reported audit count.

D Build and Runtime Verification

D.1 Build Verification

Following the security modifications and dependency update, the application was compiled using:

```
npm run build
```

The build completed successfully without TypeScript compilation errors.

D.2 Runtime Health Verification

The backend was started after remediation and tested using:

```
curl http://localhost:5000/api/health
```

Observed response:

```
{"success":true,"message":"SupplyFlow ERP API is running"}
```

This confirmed that the backend remained operational after the security changes.

D.3 Final Verification Summary

Verification	Result
JWT authentication	PASS
Invalid JWT rejection	PASS
Role-based authorization	PASS
Input validation	PASS
Business-state validation	PASS
CORS assessment	PASS
HTTP security headers	REMEDIATED
API response minimization	REMEDIATED
Dependency assessment	COMPLETED
TypeScript build	PASS
Runtime health check	PASS

Table D.1: Final Security Verification Summary